АВТОНОМНАЯ НЕКОММЕРЧЕСКАЯ ОРГАНИЗАЦИЯ ВЫСШЕГО ОБРАЗОВАНИЯ «МОСКОВСКИЙ УНИВЕРСИТЕТ “СИНЕРГИЯ”»

Факультет / Институт	Кафедра ИМиИКТ им. В. В. Дика
Направление подготовки	Информационные технологии и программирование
Форма обучения	Очная

Отчет по практической работе №9

На тему	Разработка и использование библиотек
По дисциплине	Инструментальные средства разработки программного обеспечения

Обучающийся	Ульченков Егор Витальевич	
Группа	VДКИП-111прог	

Преподаватель	Сибирев И. В.	

Москва 2026 г.

Содержание

Цель работы.....2
Требования к выполнению.....2
Реализация.....2
Интерфейс приложения.....2
Библиотека вычислений.....4
Тестовый запуск.....6
Выводы о проделанной работе.....7
Приложения.....7

Цель работы

Разработка модульного проекта на основе решения из Лабораторной работы №1. (см. приложение 1.) с использованием библиотек

Требования к выполнению

Итоговое приложение должно быть разделено на обособленные модули (части):

- Интерфейс приложения (оставить без изменений)
- Подключаемая библиотека, содержащая функции вычислений

Каждый модуль реализован в виде отдельного .NET проекта.

Реализация

Интерфейс приложения

Приложение было переработано для обеспечения соответствия требованиям модульности, расчеты вынесены во внешний проект. Используется технология DLL.

MainWindow.xaml (минимальные изменения)

```
<Window x:Class="MatrixCalc.UI.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Вычисление вектора B" Height="450" Width="800">
    <DockPanel Margin="10">
        <StackPanel Orientation="Horizontal" DockPanel.Dock="Top"
Margin="0,0,0,10">
            <TextBlock Text="N:" VerticalAlignment="Center"/>
            <TextBox x:Name="tbN" Width="40" Text="3" Margin="5,0"/>
            <TextBlock Text="M:" VerticalAlignment="Center"
Margin="10,0,0,0"/>
            <TextBox x:Name="tbM" Width="40" Text="3" Margin="5,0"/>
            <Button Content="Создать" Click="CreateMatrix_Click"
Margin="10,0,5,0"/>
            <Button Content="Заполнить случайно" Click="FillRandom_Click"
Margin="5,0"/>
            <Button Content="Рассчитать" Click="Calculate_Click"
Margin="5,0"/>
            <Button Content="Очистить" Click="Clear_Click" Margin="5,0"/>
        </StackPanel>
```

```
        <Grid>
            <Grid.ColumnDefinitions>
                <ColumnDefinition Width="*"/>
                <ColumnDefinition Width="*"/>
            </Grid.ColumnDefinitions>
            <DataGrid x:Name="dgMatrix" AutoGenerateColumns="False"
Margin="0,0,10,0"
                      CanUserAddRows="False" CanUserDeleteRows="False"
HeadersVisibility="Column"/>
            <StackPanel Grid.Column="1">
                <TextBlock Text="Результаты:" FontWeight="Bold"
Margin="0,0,0,5"/>
                <ListBox x:Name="lbResults"
ScrollViewer.HorizontalScrollBarVisibility="Auto"/>
            </StackPanel>
        </Grid>
    </DockPanel>
</Window>
```

MainWindow.xaml.cs (использованы функции из внешней DLL)

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Data;
using MatrixCalc.Core;

namespace MatrixCalc.UI
{
    public partial class MainWindow : Window
    {
        private int _n, _m;
        private List<double[]> _rows;

        public MainWindow() => InitializeComponent();

        private void CreateMatrix_Click(object sender, RoutedEventArgs e)
        {
            if (!int.TryParse(tbN.Text, out _n) || !int.TryParse(tbM.Text,
out _m) || _n < 1 || _m < 1)
            { MessageBox.Show("Введите корректные размеры N, M >= 1.");
return; }

            _rows = new List<double[]>();
            for (int i = 0; i < _n; i++) _rows.Add(new double[_m]);

            dgMatrix.Columns.Clear();
            for (int j = 0; j < _m; j++)
                dgMatrix.Columns.Add(new DataGridTextColumn
                { Header = $"Столбец{j + 1}", Binding = new Binding($"[{j}]")
{ Mode = BindingMode.TwoWay, StringFormat = "F2" } });

            dgMatrix.ItemsSource = _rows;
            lbResults.Items.Clear();
        }

        private void FillRandom_Click(object sender, RoutedEventArgs e)
        {
```

```
            if (_rows == null) { MessageBox.Show("Сначала создайте
матрицу."); return; }
            var rnd = new Random();
            foreach (var row in _rows)
                for (int j = 0; j < _m; j++)
                    row[j] = Math.Round(rnd.NextDouble() * 20 - 10, 2);
            dgMatrix.Items.Refresh();
        }

        private void Calculate_Click(object sender, RoutedEventArgs e)
        {
            if (_rows == null || _rows.Count != _n)
{ MessageBox.Show("Сначала создайте матрицу."); return; }
            dgMatrix.CommitEdit();

            var result = MatrixCalculator.Calculate(_rows.ToArray());

            lbResults.Items.Clear();
            AddResult("1. Произведение строк:", result.RowProducts.Select(v
=> v.ToString("G4")));
            AddResult("2. Среднее столбцов:", result.ColumnAverages.Select(v
=> v.ToString("G4")));
            AddResult("3. Разность max-min строк:",
result.RowDiffMinMax.Select(v => v.ToString("G4")));
            AddResult("4. Первые отрицательные столбцов:",
result.FirstNegativesInColumns.Select(v => v.HasValue ?
v.Value.ToString("G4") : "нет"));
        }

        private void Clear_Click(object sender, RoutedEventArgs e)
        {
            tbN.Text = tbM.Text = "3";
            dgMatrix.ItemsSource = null;
            dgMatrix.Columns.Clear();
            _rows = null;
            lbResults.Items.Clear();
        }

        private void AddResult(string title, IEnumerable<string> values)
        {
            lbResults.Items.Add(new TextBlock { Text = title, FontWeight =
FontWeights.Bold, Margin = new Thickness(0, 5, 0, 2) });
            lbResults.Items.Add(new TextBlock { Text = " " + string.Join(" |
", values), Margin = new Thickness(10, 0, 0, 5) });
        }
    }
}
```

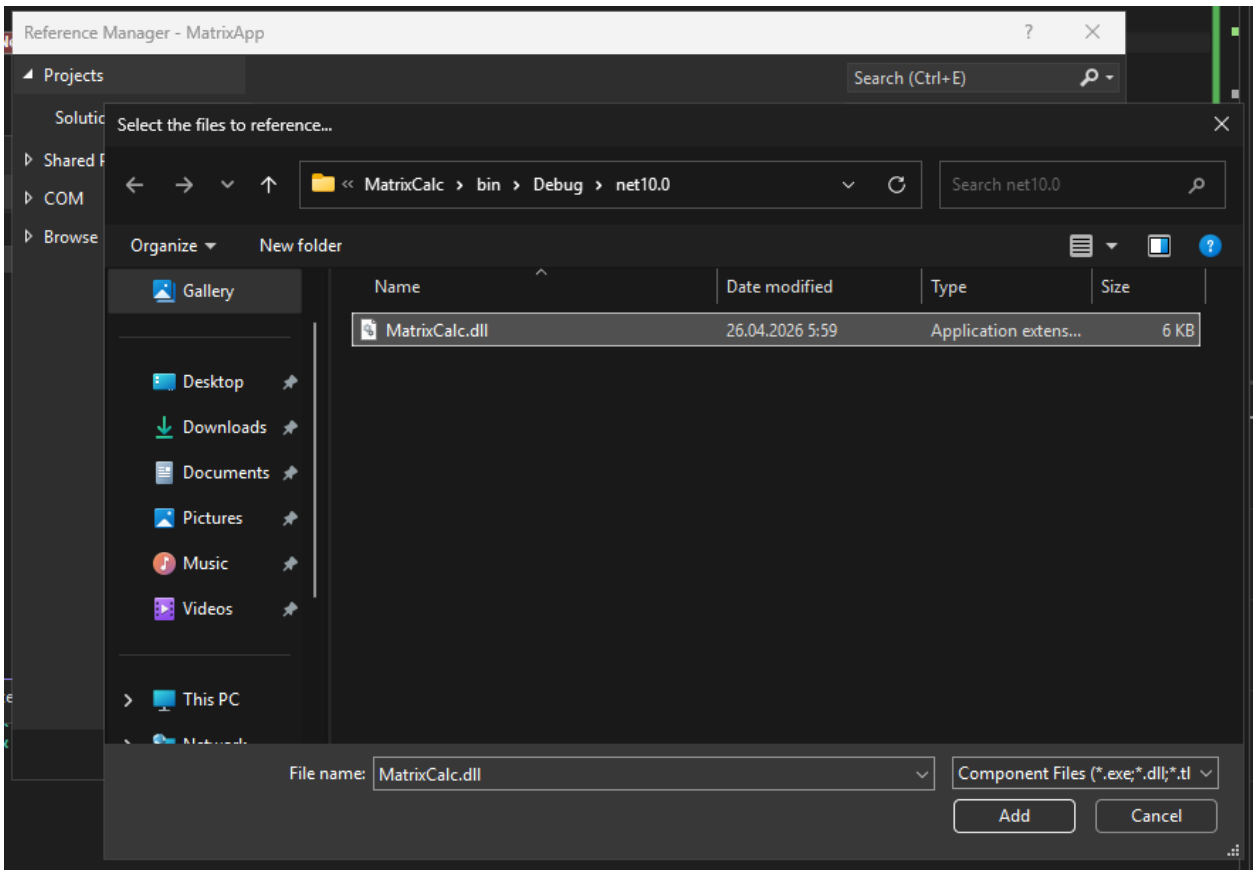
Библиотека вычислений

Функционал вычислений реализован подключаемой библиотекой

MatrixCalc/Class1.cs

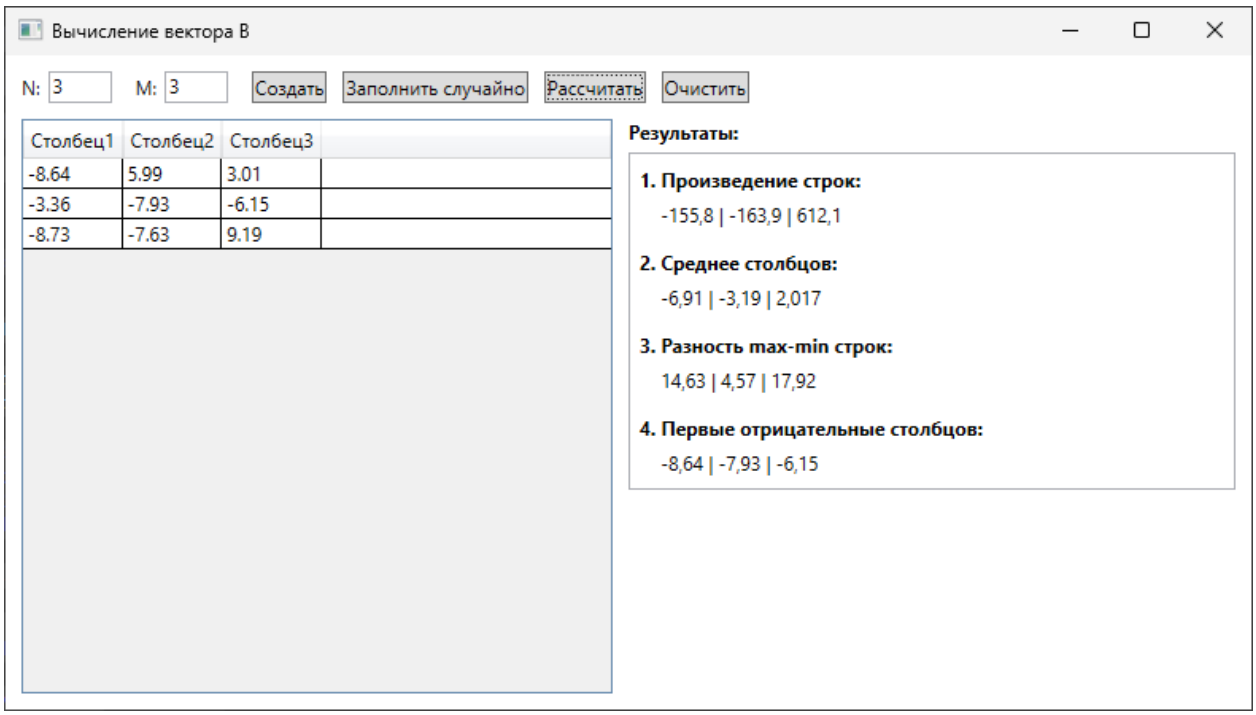
```
using System;
using System.Linq;

namespace MatrixCalc.Core
{
```

```
    public class MatrixResult
    {
        public double[] RowProducts { get; init; }
        public double[] ColumnAverages { get; init; }
        public double[] RowDiffMinMax { get; init; }
        public double?[] FirstNegativesInColumns { get; init; }
    }

    public static class MatrixCalculator
    {
        public static MatrixResult Calculate(double[][] matrix)
        {
            int n = matrix.Length;
            int m = matrix[0].Length;

            double[] b1 = new double[n];
            double[] b2 = new double[m];
            double[] b3 = new double[n];
            double?[] b4 = new double?[m];

            for (int i = 0; i < n; i++)
            {
                double prod = 1, min = matrix[i][0], max = matrix[i][0];
                foreach (double v in matrix[i])
                {
                    prod *= v;
                    if (v < min) min = v;
                    if (v > max) max = v;
                }
                b1[i] = prod;
                b3[i] = max - min;
            }

            for (int j = 0; j < m; j++)
            {
                double sum = 0;
                b4[j] = null;
                for (int i = 0; i < n; i++)
                {
                    sum += matrix[i][j];
                    if (b4[j] == null && matrix[i][j] < 0) b4[j] = matrix[i]
[j];
                }
                b2[j] = sum / n;
            }

            return new MatrixResult
            {
                RowProducts = b1,
                ColumnAverages = b2,
                RowDiffMinMax = b3,
                FirstNegativesInColumns = b4
            };
        }
    }
}
```



Библиотека была добавлена в основной проект

Тестовый запуск



Поведение приложения не изменилось, разделение на модули успешно.

Выводы о проделанной работе

В ходе выполнения работы №9 монолитное приложение успешно разделено на модульную архитектуру:

- Созданы два независимых .NET-проекта: MatrixCalc.UI (интерфейс) и MatrixCalc.Core (библиотека вычислений).
- Алгоритмы расчёта вынесены в отдельную DLL, что обеспечивает повторное использование кода, изоляцию логики и упрощение тестирования.
- Интерфейс сохранён без изменений, но теперь взаимодействует с ядром через строго типизированный метод MatrixCalculator.Calculate.
- Тестовый запуск подтвердил полную идентичность результатов и стабильность работы после рефакторинга.

Итог: закреплены навыки создания многопроектных решений в Visual Studio, управления проектными ссылками и применения принципа разделения ответственности (Separation of Concerns). Требования работы выполнены в полном объёме.

Приложения

1. Лабораторная работа №1 - Разработка структуры проекта (01 - Разработка структуры проекта.pdf)